lunadem

v0.3.0

Official Python Library Documentation

Lunar Digital Elevation Model Generation Toolkit

Author & Library Developer

Kartavya Mahesh Suryawanshi

github.com/Kartavya728

Documentation Author: Kartavya Mahesh Suryawanshi

2026-04-13

Generation of High-Resolution Digital Elevation Models from Lunar Images using Photoclinometry

Contents

1 Introduction **3**
- 1.1 Overview . . . 3
- 1.2 Key Capabilities . . . 3
- 1.3 Repository & Credits . . . 4

2 Installation **5**
- 2.1 Requirements . . . 5
- 2.2 Installation Variants . . . 5
- 2.3 Verifying the Installation . . . 5

3 Quick-Start Guide **6**
- 3.1 Download the Test Scene . . . 6
- 3.2 Generate a DEM . . . 6
- 3.3 Load a Kaguya Scene and Find a Landing Site . . . 6
- 3.4 Predict Scene Metadata . . . 7

4 CLI Reference **8**

5 Core API Reference **9**
- 5.1 `generate_dem` . . . 9
- 5.2 `analyze_dem` . . . 10
- 5.3 `assess_landing` . . . 11

6 Dataset & Scene Loading API **13**
- 6.1 `load_kaguya_scene` . . . 13
- 6.2 `build_scene_summary` . . . 14
- 6.3 Scene Geometry Accessors . . . 14

7 Landing Site API **16**
- 7.1 `find_safe_landing_site` . . . 16
- 7.2 `get_rover_spec` & `list_available_rovers` . . . 18

8 ML Inference API **19**
- 8.1 Overview . . . 19
- 8.2 Bundled Model Metrics . . . 19
- 8.3 Inference Functions . . . 19
- 8.4 `get_model_metrics` & `list_model_names` . . . 20

9 Geometry & Conversion API **21**
9.1 Coordinate Conversions . . . 21
9.2 Area & Coverage Helpers . . . 21
9.3 Lighting . . . 22

10 Visualization API **23**
10.1 Backend Options . . . 23
10.2 Surface Plots . . . 23
10.3 Scene Geometry Plots . . . 24
10.4 Landing-Site Plots . . . 24

11 Configuration Classes **26**
11.1 `ReconstructionConfig` . . . 26
11.2 Configuration Class Summary . . . 27

12 Data Models (Return Types) **28**

13 I/O & File Utilities **29**
13.1 Image Loading . . . 29
13.2 DEM Export . . . 29
13.3 PDS Labels . . . 29
13.4 Manifests & Utilities . . . 30

14 Constants **31**

15 SFS Theory Reference **32**
15.1 Accessing Bundled Theory Documents . . . 32
15.2 Brief SFS Theory . . . 32

16 Troubleshooting **33**

17 Release & Development **34**
17.1 Release Checklist . . . 34
17.2 Development Install . . . 34

Appendix A — Full Function Index **35**

1. Introduction

1.1 Overview

lunadem is an open-source Python library for generating high-resolution Digital Elevation Models (DEMs) from single lunar images using shape-from-shading (SFS) photoclinometry techniques. Developed by **Kartavya Mahesh Suryawanshi**, it provides an end-to-end pipeline from raw Kaguya Terrain Camera imagery to metric DEM products, terrain analytics, rover landing-site assessment, and interactive 3-D visualization.

> **Tip**
>
> **lunadem** is ideal for planetary scientists, aerospace engineers, and researchers working on lunar surface characterization, rover mission planning, and remote sensing algorithm development.

1.2 Key Capabilities

- **DEM Generation** — SFS, multi-scale SFS, ML baseline, and hybrid blending methods.
- **Kaguya Scene Loader** — STAC + camera metadata ingestion with full geometry derivation.
- **Terrain Analytics** — Slope, roughness, and curvature maps with configurable parameters.
- **Landing-Site Assessment** — Rover-aware safety scoring and visualization.
- **Offline ML Prediction** — Bundled ONNX models for illumination and view-geometry estimation.
- **Dual Plotting Backends** — Matplotlib (static) and Plotly (interactive hoverable 3-D).
- **Export Formats** — GeoTIFF, Wavefront OBJ, PLY, JSON manifests.

1.3 Repository & Credits

Field	Value
Library name	`lunadem`
Version	`0.3.0`
Author	Kartavya Mahesh Suryawanshi
GitHub	https://github.com/Kartavya728
Canonical import	`import lunadem`
Legacy import	`import lunardem` (deprecated)
Moon radius used	`1 737 400.0 m`

2. Installation

2.1 Requirements

lunadem requires Python 3.9 or later. All scientific dependencies (NumPy, SciPy, Pillow, rasterio, *etc.*) are pulled in automatically via the base install.

2.2 Installation Variants

Base Install

```
pip install lunadem
```

ML Extras (model retraining / export)

```
pip install "lunadem[ml]"
```

Visualization Extras

```
pip install "lunadem[viz]"
```

Development Extras (tests

```
pip install "lunadem[dev]"
```

Full Install

```
pip install "lunadem[ml,viz,dev]"
```

2.3 Verifying the Installation

```python
import lunadem
print(lunadem.__version__)    # Expected: 0.3.0
```

Listing 2.1: Verify lunadem installation

3. Quick-Start Guide

3.1 Download the Test Scene

```python
from lunadem import download_test_scene

downloads = download_test_scene('downloads', overwrite=False)
print(downloads)
# {'image': '.../.../image.tif', 'item': '.../.../item.json', ...}
```

Listing 3.1: Download the packaged reference scene

3.2 Generate a DEM

```python
from lunadem import ReconstructionConfig, generate_dem

# Build a typed configuration (all fields have sensible defaults)
config = ReconstructionConfig(
    output={'output_dir': 'output', 'base_name': 'my_dem'}
)

result = generate_dem(
    'dataset/TC1S2B0_01_07496N087E3020/image/image.tif',
    method='hybrid',
    config=config,
)

print(result.dem_meters.shape)    # e.g. (512, 512)
print(result.pixel_scale_m)       # ground sample distance in metres
```

Listing 3.2: End-to-end DEM generation with the hybrid method

3.3 Load a Kaguya Scene and Find a Landing Site

```python
from lunadem import (
    load_kaguya_scene,
    get_rover_spec,
    find_safe_landing_site,
    plot_landing_site_2d,
)

# Load the packaged reference scene
```

```python
scene = load_kaguya_scene('TC1S2B0_01_07496N087E3020')
print(scene.scene_id)           # TC1S2B0_01_07496N087E3020
print(scene.image_path)         # Path to image.tif

# Retrieve a built-in rover specification
rover = get_rover_spec('pragyan')

# Find the safest landing site
landing = find_safe_landing_site(
    scene.image_path,
    rover=rover,
    scene=scene,
    method='hybrid',
)
print(landing.summary)

# Save 2-D overlay (PNG + interactive HTML)
plot_landing_site_2d(
    scene.image_path,
    landing,
    backend='both',
    show=False,
    save_path='output/landing_2d.png',
    html_path='output/landing_2d.html',
)
```

Listing 3.3: Load scene, score landing site, and visualize

3.4 Predict Scene Metadata

```python
from lunadem import predict_scene_metadata

pred = predict_scene_metadata(
    'dataset/TC1S2B0_01_07496N087E3020/image/image.tif',
    max_patches=8,
)
print(pred.targets)      # {'sun_azimuth_deg': 101.3,
    'sun_elevation_deg': 34.8, ...}
print(pred.patch_count)  # 8
```

Listing 3.4: Run all bundled ONNX models on a TIFF

4. CLI Reference

All commands are invoked as `lunadem <command> [options]`.

Command	Purpose
`generate`	Reconstruct a DEM from an input image or TIFF.
`analyze`	Compute terrain metrics from an existing DEM.
`landing`	Assess landing safety from an existing DEM.
`scene-summary`	Summarize STAC + camera metadata and derive geometry.
`predict`	Run the bundled metadata estimators.
`sfs -maths`	Print the SFS mathematics reference.
`sfs -terms`	Print the SFS terminology reference.
`sfs -assumptions`	Print the SFS assumptions reference.
`plot-surface`	Visualize a local terrain surface in 3-D.
`plot-scene`	Visualize Moon, footprint, camera, Sun, and Earth geometry.
`landing-site`	Locate the safest rover landing site and plot it.
`download -test`	Download the official packaged test scene.

5. Core API Reference

5.1 generate_dem

```
lunadem.core.api
```

Signature

```python
generate_dem(
    input_data: np.ndarray | str | Path,
    method: str = 'sfs',
    config: ReconstructionConfig | Dict[str, Any] | None = None,
    analysis_config: AnalysisConfig | Dict[str, Any] | None = None,
) -> DEMResult
```

Generate a Digital Elevation Model from a single lunar image. Supports four reconstruction methods: classical SFS, multi-scale SFS, ML baseline, and hybrid blending.

Parameters

Parameter	Type	Default	Description
`input_data`	`np.ndarray | str | Path`	*required*	Input image array or path to a PNG/JPG/TIFF/GeoTIFF file.
`method`	`str`	`'sfs'`	Reconstruction algorithm. One of `'sfs'`, `'multiscale_sfs'`, `'ml'`, `'hybrid'`.
`config`	`ReconstructionConfig | dict | None`	`None`	Full reconstruction configuration. Accepts a typed model or a plain dictionary. Defaults are used when `None`.
`analysis_config`	`AnalysisConfig | dict | None`	`None`	Terrain analysis configuration applied after reconstruction.

Returns

Returns a `DEMResult` dataclass containing:

- `dem_meters` — float32 NumPy array of height values in metres.
- `method` — name of the method used.
- `pixel_scale_m` — ground sample distance.
- `exports` — paths to saved GeoTIFF/OBJ/PLY/HTML files.
- `metrics` — optional `TerrainMetrics` object.

Example

```python
from lunadem import ReconstructionConfig, generate_dem

config = ReconstructionConfig(
    output={'output_dir': 'output', 'base_name': 'demo'}
)

result = generate_dem(
    'dataset/TC1S2B0_01_07496N087E3020/image/image.tif',
    method='hybrid',
    config=config,
)
print(result.dem_meters.shape)    # (H, W)
print(result.pixel_scale_m)       # e.g. 4.79 m
```

Listing 5.1: Generate a DEM using the hybrid method

> **Note**
> Prefer typed config models (`ReconstructionConfig`) over plain dictionaries when you want explicit parameter validation before execution.

5.2 analyze_dem

`lunadem.core.api`

Signature

```python
analyze_dem(
    dem_or_path: np.ndarray | str | Path,
    analysis_config: AnalysisConfig | Dict[str, Any] | None = None,
) -> TerrainMetrics
```

Compute terrain metrics (slope, roughness, curvature) from an existing DEM.

Parameters

Parameter	Type	Default	Description
dem_or_path	np.ndarray | str | Path	*required*	A height-map array or path to a previously saved DEM file.
analysis_config	AnalysisConfig | dict | None	None	Terrain analysis settings (pixel size, roughness window, histogram bins).

Returns

A `TerrainMetrics` object with fields `slope_deg`, `roughness_m`, `curvature`, `stats`, and `histograms`.

Example

```python
from lunadem import AnalysisConfig, analyze_dem

config = AnalysisConfig(pixel_size_m=5.0, roughness_window=7)
metrics = analyze_dem('output/demo_dem.tif', analysis_config=config)

print(metrics.stats['mean_slope_deg'])
print(metrics.stats['mean_roughness_m'])
```

Listing 5.2: Compute terrain metrics from a saved DEM

5.3 assess_landing

`lunadem.core.api`

Signature

```python
assess_landing(
    dem_or_path: np.ndarray | str | Path,
    landing_config: LandingConfig | Dict[str, Any] | None = None,
    analysis_config: AnalysisConfig | Dict[str, Any] | None = None,
) -> LandingReport
```

Assess the landing suitability of each pixel in a DEM. Returns slope, roughness, and hazard masks together with a composite safety score.

Parameters

Parameter	Type	Default	Description
`dem_or_path`	`np.ndarray | str | Path`	*required*	Height-map array or path to a DEM file.
`landing_config`	`LandingConfig | dict | None`	`None`	Landing constraint thresholds (max slope, roughness, hazard height, etc.).
`analysis_config`	`AnalysisConfig | dict | None`	`None`	Terrain analysis settings used during the assessment.

Returns

A `LandingReport` with `safe_mask`, `slope_mask`, `roughness_mask`, `hazard_mask`, `safe_fraction`, and `score`.

Example

```python
from lunadem import LandingConfig, LandingConstraints, assess_landing

constraints = LandingConstraints(max_slope_deg=10.0,
    max_roughness_m=1.0)
cfg = LandingConfig(pixel_size_m=5.0, constraints=constraints)

report = assess_landing('output/demo_dem.tif', landing_config=cfg)
print(f"Safe fraction: {report.safe_fraction:.2%}")
print(f"Overall score: {report.score:.4f}")
```

Listing 5.3: Assess landing suitability

6. Dataset & Scene Loading API

6.1 load_kaguya_scene

```
lunadem.datasets.kaguya
```

Signature

```
load_kaguya_scene(
    scene_id_or_path: str | Path = 'TC1S2B0_01_07496N087E3020',
    dataset_root: str | Path = 'dataset',
) -> KaguyaScene
```

Load a complete local Kaguya scene bundle, including the STAC item, camera model, and image path.

Parameters

Parameter	Type	Default	Description
scene_id_or_path	str | Path	'TC1S2B0_...3020'	Scene identifier string or path to a local scene directory.
dataset_root	str | Path	'dataset'	Root directory under which scene subdirectories are stored.

Returns

A `KaguyaScene` dataclass with fields: `scene_id`, `scene_dir`, `image_path`, `stac_item`, `camera_model`, `geometry`.

Example

```
from lunadem import load_kaguya_scene

scene = load_kaguya_scene('TC1S2B0_01_07496N087E3020')
print(scene.scene_id)
print(scene.image_path)
print(scene.geometry.centroid_lat_deg)
```

Listing 6.1: Load the packaged test scene

6.2 build_scene_summary

```
lunadem.datasets.kaguya
```

Signature

```python
build_scene_summary(
    scene_or_geometry_or_item: KaguyaScene | SceneGeometry | Mapping
        | str | Path,
    camera_model: Mapping | str | Path | None = None,
) -> Dict[str, Any]
```

Build a JSON-serializable summary of a scene's metadata and all derived geometric quantities.

Example

```python
from lunadem import build_scene_summary, load_kaguya_scene

scene = load_kaguya_scene('TC1S2B0_01_07496N087E3020')
summary = build_scene_summary(scene)

print(summary['centroid'])            # [lat, lon]
print(summary['ground_area_km2'])
```

Listing 6.2: Build and print a scene summary

6.3 Scene Geometry Accessors

The following convenience functions all share the same first argument (`scene_or_geometry_or_item` : a scene ID, directory path, `KaguyaScene` , or raw STAC mapping) and an optional `camera_model` argument.

Function	Returns	Description
`get_scene_centroid`	`tuple[float|None, float|None]`	Centroid lat/lon in degrees.
`get_scene_bbox`	`tuple[float,...] | None`	Geographic bounding box.
`get_scene_footprint_lonlat`	`list[tuple[float,float]]`	Footprint polygon in lon/lat.
`get_scene_footprint_xyz`	`list[tuple[float,`	Footprint in Moon-centered XYZ.
`get_scene_ground_area_km2`	`float | None`	Ground area in km^2.

Function	Returns	Description
get_scene_ground_coverage	tuple[float|None, float|None]	(height_m, width_m) coverage.
get_scene_sun_vector	np.ndarray | None	Normalized sun direction vector.
get_scene_sun_summary	Dict[str,Any]	Compact sun geometry metadata.
get_scene_camera_summary	Dict[str,Any]	Compact camera path/pose summary.
get_scene_view_angles	Dict[str,float|No	View-angle metadata in degrees.
get_scene_transform	tuple[float,...] | None	GDAL projected transform tuple.
get_scene_time_range	tuple[str|None, str|None]	Acquisition time range (ISO strings).

7. Landing Site API

7.1 find_safe_landing_site

```
lunadem.landing.sites
```

Signature

```python
find_safe_landing_site(
    image_or_dem: np.ndarray | str | Path,
    *,
    rover: str | RoverSpec | Mapping | None = None,
    scene: Any = None,
    sun: Any = None,
    camera: Mapping | None = None,
    time: str | None = None,
    method: str = 'hybrid',
    reconstruct: bool | None = None,
    config: ReconstructionConfig | Mapping | None = None,
    analysis_config: AnalysisConfig | Mapping | None = None,
    landing_config: LandingConfig | Mapping | None = None,
) -> LandingSiteResult
```

Identify the globally safest rover landing site on an image or pre-computed DEM. When given a raw image and `reconstruct=True` (or auto-inferred), the function first reconstructs a DEM then scores every pixel.

Parameters

Parameter	Type	Default	Description
`image_or_dem`	`np.ndarray | str | Path`	*required*	A NumPy array, image path, or TIFF path. Can be a raw image (DEM is reconstructed) or a pre-built DEM.
`rover`	`str | RoverSpec | Mapping | None`	`None`	Rover specification by name (`'pragyan'`, `'yutu'`, etc.) or a custom `RoverSpec`.

Parameter	Type	Default	Description
scene	Any	None	KaguyaScene, scene ID, or directory. Provides illumination metadata when available.
sun	Any	None	Override sun geometry. If omitted, derived from scene.
camera	Mapping | None	None	Camera model override.
time	str | None	None	ISO timestamp for acquisition time.
method	str	'hybrid'	Reconstruction method ('sfs', 'multiscale_sfs', 'ml', 'hybrid').
reconstruct	bool | None	None	Force DEM reconstruction. None = auto-detect.
config	ReconstructionConfig | Mapping | None	None	Reconstruction configuration.
analysis_config	AnalysisConfig | Mapping | None	None	Terrain analysis configuration.
landing_config	LandingConfig | Mapping | None	None	Landing constraint thresholds.

Returns

A LandingSiteResult with fields row, col, score, x_m, y_m, safe_radius_px, rover, safe_mask, score_map, and summary.

Example

```python
from lunadem import (
    load_kaguya_scene,
    get_rover_spec,
    find_safe_landing_site,
    plot_landing_site_3d,
)

scene  = load_kaguya_scene('TC1S2B0_01_07496N087E3020')
rover  = get_rover_spec('pragyan')

landing = find_safe_landing_site(
    scene.image_path,
    rover=rover,
    scene=scene,
)
```

```python
print(landing.summary)

plot_landing_site_3d(
    scene.image_path,
    landing,
    backend='both',
    show=False,
    save_path='output/landing_3d.png',
    html_path='output/landing_3d.html',
)
```

Listing 7.1: Full landing-site pipeline

7.2 get_rover_spec & list_available_rovers

```python
from lunadem import list_available_rovers, get_rover_spec

rovers = list_available_rovers()
print(rovers)  # ['curiosity', 'perseverance', 'pragyan', 'yutu',
    ...]

# Retrieve a preset
spec = get_rover_spec('pragyan')
print(spec.length_m, spec.width_m, spec.ground_clearance_m)

# Or define a custom rover
custom = get_rover_spec(
    length_m=1.5,
    width_m=0.9,
    height_m=0.8,
    ground_clearance_m=0.3,
    safety_margin_m=0.5,
)
```

Listing 7.2: List all built-in rover presets

8. ML Inference API

8.1 Overview

lunadem bundles three ONNX convolutional neural networks for offline metadata prediction. No internet connection is required. All models accept image patches extracted from the input TIFF.

8.2 Bundled Model Metrics

Model	Targets	MAE	Unit
`illumination_cnn`	`sun_azimuth_deg`	1.608062	degrees
	`sun_elevation_deg`	0.002216	degrees
`view_geometry_cnn`	`off_nadir_deg`	0.105210	degrees
	`view_azimuth_deg`	0.051268	degrees
`scene_context_cnn`	`gsd_m`	0.082854	metres
	`centroid_lat_deg`	3.149079	degrees
	`centroid_lon_deg`	0.509205	degrees

Trained on 18 scenes × 16 patches. Package note: `real_onnx_models`.

8.3 Inference Functions

```python
from lunadem import predict_scene_metadata

pred = predict_scene_metadata(
    'dataset/TC1S2B0_01_07496N087E3020/image/image.tif',
    max_patches=16,
)
print(pred.targets)
# {'sun_azimuth_deg': 101.4, 'sun_elevation_deg': 34.8,
#  'off_nadir_deg': 0.12, 'view_azimuth_deg': 93.2,
#  'gsd_m': 4.79, 'centroid_lat_deg': 8.7, 'centroid_lon_deg': 302.0}
```

Listing 8.1: Run all metadata models at once

```python
from lunadem import (
    predict_illumination,
    predict_view_geometry,
    predict_scene_location,
```

```
)

illum  = predict_illumination('image.tif')
view   = predict_view_geometry('image.tif')
loc    = predict_scene_location('image.tif')

print(illum.targets)
print(view.targets)
print(loc.targets)
```

Listing 8.2: Run individual models

8.4 get_model_metrics & list_model_names

```
from lunadem import get_model_metrics, list_model_names,
    get_model_targets

print(list_model_names())
# ['illumination_cnn', 'scene_context_cnn', 'view_geometry_cnn']

metrics = get_model_metrics()

targets = get_model_targets('illumination_cnn')
print(targets)  # ('sun_azimuth_deg', 'sun_elevation_deg')
```

Listing 8.3: Inspect bundled model information

9. Geometry & Conversion API

9.1 Coordinate Conversions

```python
from lunadem import (
    latlon_to_cartesian_moon,
    cartesian_to_latlon_moon,
    deg_to_rad,
    rad_to_deg,
    pixel_to_projected,
    projected_to_pixel,
)

# Lat/lon -> Moon-centered XYZ (metres)
x, y, z = latlon_to_cartesian_moon(lat_deg=8.7, lon_deg=302.0)

# Reverse
lat, lon, r = cartesian_to_latlon_moon(x, y, z)

# Angle conversions
r = deg_to_rad(45.0)    # 0.7854
d = rad_to_deg(r)       # 45.0

# Pixel <-> projected  (GDAL 6-element transform tuple)
transform = (302.0, 0.0001, 0.0, 8.7, 0.0, -0.0001)
px, py = pixel_to_projected(row=100, col=200, transform=transform)
row, col = projected_to_pixel(x=px, y=py, transform=transform)
```

Listing 9.1: Coordinate conversion examples

9.2 Area & Coverage Helpers

```python
from lunadem import (
    bbox_area_deg2,
    bbox_area_km2,
    ground_coverage_meters,
)

bbox  = [301.9, 8.6, 302.1, 8.8]    # [lon_min, lat_min, lon_max,
    lat_max]
area_deg2 = bbox_area_deg2(bbox)
area_km2  = bbox_area_km2(bbox, lat_center_deg=8.7)

h_m, w_m = ground_coverage_meters(shape=(512, 512), gsd_m=4.79)
```

Listing 9.2: Bounding-box area and ground coverage

9.3 Lighting

```python
from lunadem import get_light_vector, calculate_surface_normals, calculate_predicted_image
import numpy as np

light = get_light_vector(sun_azimuth_deg=101.55, sun_elevation_deg=34.80)
# array([...])  shape (3,)      +X east, +Y north, +Z up

# Lambertian reflectance from a DEM
dem = np.random.rand(128, 128).astype('float32')
normals = calculate_surface_normals(dem)        # (128, 128, 3)
predicted = calculate_predicted_image(dem, light)  # (128, 128)
```

Listing 9.3: Compute a unit light vector from sun angles

10. Visualization API

10.1 Backend Options

All visualization functions accept a `backend` keyword argument:

Value	Engine	Output
`'matplotlib'`	Matplotlib	Static PNG / interactive window
`'plotly'`	Plotly	HTML with hover & rotate
`'both'`	Both	Saves both PNG and HTML

Note

Always pass `show=False` in scripts and CI pipelines. Use `show=True` only in interactive sessions.

10.2 Surface Plots

```python
from lunadem import plot_depth_map, plot_3d_surface,
    plot_3d_surface_interactive

# 2-D colour map (Matplotlib)
plot_depth_map(
    dem,
    save_path='output/dem_2d.png',
    title='Lunar DEM',
    show=False,
)

# 3-D surface (Matplotlib)
plot_3d_surface(
    dem,
    save_path='output/dem_3d.png',
    show=False,
)

# Interactive 3-D (Plotly)
fig = plot_3d_surface_interactive(
    dem,
    title='Interactive Lunar Surface',
    save_path='output/dem_3d.html',
    show=False,
)
```

Listing 10.1: Plot a reconstructed DEM

10.3 Scene Geometry Plots

```
from lunadem import load_kaguya_scene, plot_scene_geometry_3d,
    plot_moon_surface_3d

scene = load_kaguya_scene('TC1S2B0_01_07496N087E3020')

# Moon globe + footprint + camera + Sun + Earth
plot_scene_geometry_3d(
    scene,
    backend='both',
    show=False,
    save_path='output/scene_geometry.png',
    html_path='output/scene_geometry.html',
)

# Moon surface + footprint only
plot_moon_surface_3d(
    scene,
    backend='plotly',
    show=False,
    html_path='output/moon_surface.html',
)
```

Listing 10.2: Visualize Moon globe and scene footprint

10.4 Landing-Site Plots

```
from lunadem import plot_landing_site_2d, plot_landing_site_3d

plot_landing_site_2d(
    scene.image_path,
    landing,
    backend='both',
    show=False,
    save_path='output/landing_2d.png',
    html_path='output/landing_2d.html',
)

plot_landing_site_3d(
    scene.image_path,
    landing,
    backend='both',
    show=False,
```

```
    save_path='output/landing_3d.png',
    html_path='output/landing_3d.html',
)
```

Listing 10.3: Landing-site 2-D and 3-D overlays

11. Configuration Classes

11.1 ReconstructionConfig

Top-level configuration object. All sub-configs have sensible defaults — only override what you need.

```python
from lunadem import (
    ReconstructionConfig,
    IlluminationConfig,
    SensorConfig,
    SFSMethodConfig,
    HybridMethodConfig,
    OutputConfig,
)

config = ReconstructionConfig(
    illumination=IlluminationConfig(
        sun_azimuth_deg=101.55,
        sun_elevation_deg=34.80,
    ),
    sensor=SensorConfig(
        spacecraft_altitude_km=95.85,
        focal_length_mm=140.0,
        detector_pixel_width_um=7.0,
    ),
    sfs=SFSMethodConfig(
        max_iterations=200,
        regularization_lambda=0.003,
        multiscale_levels=4,
    ),
    hybrid=HybridMethodConfig(
        blend_weight=0.65,
        fallback_to_sfs_on_ml_failure=True,
    ),
    output=OutputConfig(
        output_dir='output',
        base_name='my_dem',
        save_geotiff=True,
        save_obj=True,
        save_interactive_html=True,
    ),
)
```

Listing 11.1: Full ReconstructionConfig example

11.2 Configuration Class Summary

Class	Purpose & Key Fields
`ReconstructionConfig`	Top-level config. Nests all sub-configs below.
`IlluminationConfig`	Sun azimuth/elevation in degrees (`sun_azimuth_deg`, `sun_elevation_deg`).
`SensorConfig`	Spacecraft altitude (km), focal length (mm), pixel width (µm).
`GeoreferenceConfig`	Lunar CRS string, projection name, optional corner coordinates.
`PreprocessingConfig`	Normalize, Gaussian sigma, median size, shadow masking.
`SFSMethodConfig`	Iterations, regularization λ, convergence tol, multiscale levels.
`MLMethodConfig`	Enable/disable ML baseline, model path, fallback sigma.
`HybridMethodConfig`	Blend weight (0–1), fallback-to-SFS flag.
`OutputConfig`	Output directory, base name, save flags for GeoTIFF/OBJ/PLY/HTML.
`AnalysisConfig`	Pixel size (m), roughness window size, histogram bins.
`LandingConfig`	Pixel size (m), nested `LandingConstraints`.
`LandingConstraints`	`max_slope_deg`, `max_roughness_m`, `hazard_height_m`, `hazard_window`, `min_safe_area_px`.

12. Data Models (Return Types)

Class	Key Fields
`DEMResult`	`dem_meters`, `method`, `pixel_scale_m`, `crs`, `exports`, `diagnostics`, `metrics`
`TerrainMetrics`	`slope_deg`, `roughness_m`, `curvature`, `stats`, `histograms`
`LandingReport`	`safe_mask`, `slope_mask`, `roughness_mask`, `hazard_mask`, `safe_fraction`, `score`, `summary`
`LandingSiteResult`	`row`, `col`, `score`, `x_m`, `y_m`, `safe_radius_px`, `rover`, `safe_mask`, `score_map`, `summary`
`KaguyaScene`	`scene_id`, `scene_dir`, `image_path`, `stac_item`, `camera_model`, `geometry`
`SceneGeometry`	Centroid, bbox, GSD, footprint, sun vector, camera path, view angles, transform, ground coverage
`RoverSpec`	`name`, `length_m`, `width_m`, `height_m`, `ground_clearance_m`, `safety_margin_m`
`MetadataPrediction`	`model_name`, `targets`, `uncertainty`, `metrics`, `patch_count`, `source`

13. I/O & File Utilities

13.1 Image Loading

```python
from lunadem import load_image

# Supports PNG, JPG, TIFF, GeoTIFF, and in-memory arrays
image, meta = load_image('path/to/image.tif', normalize=True)
print(image.dtype)     # float32
print(image.shape)     # (H, W)
print(meta)            # {'source': 'geotiff', 'shape': ..., 'crs':
    ...}
```

Listing 13.1: Load various image formats

13.2 DEM Export

```python
from lunadem import (
    save_dem_as_geotiff,
    save_dem_as_obj,
    save_dem_as_ply,
    GeoreferenceConfig,
)

georef = GeoreferenceConfig()

# GeoTIFF (compressed, recommended)
path = save_dem_as_geotiff('output/dem.tif', dem, georef)

# Wavefront OBJ (for 3D rendering engines)
path = save_dem_as_obj('output/dem.obj', dem)

# ASCII PLY (for point-cloud tools)
path = save_dem_as_ply('output/dem.ply', dem)
```

Listing 13.2: Export DEM to multiple formats

13.3 PDS Labels

```python
from lunadem import load_pds_label, extract_pds_geometry

label = load_pds_label('path/to/label.lbl')
```

```
geom  = extract_pds_geometry(label)
print(geom)
```

Listing 13.3: Load a PDS label and extract geometry

13.4 Manifests & Utilities

```
from lunadem import save_manifest, ensure_directory

out_dir = ensure_directory('output/run_001')
save_manifest(out_dir / 'manifest.json', {
    'scene_id': 'TC1S2B0_01_07496N087E3020',
    'method': 'hybrid',
    'config': config.model_dump(),
})
```

Listing 13.4: Save a reproducibility manifest

14. Constants

Constant	Value	Description
`MOON_RADIUS_M`	`1 737 400.0`	Mean lunar radius in metres.
`DEFAULT_MOON_CRS`	`+proj=longlat +a=1737400 ...`	Default IAU Moon CRS string.
`TEST_SCENE_ID`	`TC1S2B0_01_07496N087E3020`	Bundled reference scene identifier.
`__version__`	`0.3.0`	Library version string.

```python
from lunadem import MOON_RADIUS_M, DEFAULT_MOON_CRS, TEST_SCENE_ID,
    __version__

print(MOON_RADIUS_M)       # 1737400.0
print(TEST_SCENE_ID)       # TC1S2B0_01_07496N087E3020
print(__version__)         # 0.3.0
```

Listing 14.1: Importing constants

15. SFS Theory Reference

15.1 Accessing Bundled Theory Documents

lunadem ships three SFS theory documents as plain text, accessible programmatically or via the CLI.

```python
from lunadem import get_sfs_math, get_sfs_terms, get_sfs_assumptions

print(get_sfs_math())          # Full SFS mathematics derivation
print(get_sfs_terms())         # Terminology glossary
print(get_sfs_assumptions())   # Model assumptions
```

Listing 15.1: Access SFS theory documents

Warning

The SFS algorithm assumes Lambertian (diffuse) reflectance. Highly specular surfaces or very low sun-elevation images (<5°) may produce artefacts. See `get_sfs_assumptions()` for the complete list of model constraints.

15.2 Brief SFS Theory

Shape-from-Shading recovers a height map $z(x, y)$ by solving the irradiance equation:

$$I(x, y) = R\Big(\mathbf{n}(x, y),\ \hat{\ell}\Big)$$

where I is the observed pixel intensity, $\mathbf{n}$ is the surface normal derived from z, and $\hat{\ell}$ is the unit sun vector. Under the Lambertian reflectance model:

$$R(\mathbf{n}, \hat{\ell}) = \max(\mathbf{n} \cdot \hat{\ell},\ 0)$$

The optimization minimizes:

$$\mathcal{L}(z) = \|I - R(\nabla z, \hat{\ell})\|^2 + \lambda \|\nabla^2 z\|^2$$

where λ (`regularization_lambda`) controls smoothness.

16. Troubleshooting

Problem	Solution
Matplotlib windows pop up during scripts	Pass `show=False` and use `save_path`. In CI, set `MPLBACKEND=Agg`.
ONNX import fails / model export errors	Ensure `ml-dtypes>=0.5` is installed or use the compatibility shim in the training script.
SFS is slow on large images	Reduce `config.sfs.max_long_side_px` (default 1024).
Only need metadata, not a full DEM	Call `predict_*` APIs directly without reconstructing.
`lunardem` import warning	Update imports to `lunadem` — `lunardem` is deprecated but still functional.
GeoTIFF has no CRS info	Pass an explicit `GeoreferenceConfig` with corner coordinates.

17. Release & Development

17.1 Release Checklist

1. Run the full test suite: `pytest tests/`.
2. Re-train metadata models when the dataset changes.
3. Re-run the documentation generator.
4. Build wheel and source distribution: `python -m build`.
5. Verify ONNX artifacts are bundled in the wheel.
6. Publish to PyPI and update the GitHub release notes at https://github.com/Kartavya728.

17.2 Development Install

```
git clone https://github.com/Kartavya728/lunadem.git
cd lunadem
pip install -e ".[dev]"
pytest tests/ -v
```

Appendix A — Full Function Index

Function / Class	Source Module
acquisition_duration_seconds	lunadem.geometry.conversions
analyze_dem	lunadem.core.api
assess_landing	lunadem.core.api
bbox_area_deg2	lunadem.geometry.conversions
bbox_area_km2	lunadem.geometry.conversions
build_scene_summary	lunadem.datasets.kaguya
calculate_predicted_image	lunadem.geometry.surface
calculate_surface_normals	lunadem.geometry.surface
cartesian_to_latlon_moon	lunadem.geometry.conversions
deg_to_rad	lunadem.geometry.conversions
download_test_scene	lunadem.datasets.kaguya
ensure_directory	lunadem.utils.files
extract_pds_geometry	lunadem.io.pds
find_safe_landing_site	lunadem.landing.sites
generate_dem	lunadem.core.api
get_light_vector	lunadem.geometry.lighting
get_model_artifact_paths	lunadem.ml.inference
get_model_metrics	lunadem.ml.inference
get_model_targets	lunadem.ml.inference
get_rover_spec	lunadem.landing.rovers
get_scene_bbox	lunadem.datasets.kaguya
get_scene_camera_summary	lunadem.datasets.kaguya
get_scene_centroid	lunadem.datasets.kaguya
get_scene_footprint_lonlat	lunadem.datasets.kaguya
get_scene_footprint_xyz	lunadem.datasets.kaguya
get_scene_ground_area_km2	lunadem.datasets.kaguya
get_scene_ground_coverage	lunadem.datasets.kaguya
get_scene_sun_summary	lunadem.datasets.kaguya
get_scene_sun_vector	lunadem.datasets.kaguya
get_scene_time_range	lunadem.datasets.kaguya
get_scene_transform	lunadem.datasets.kaguya
get_scene_view_angles	lunadem.datasets.kaguya
get_sfs_assumptions	lunadem.sfs_theory

Function / Class	Source Module
get_sfs_math	lunadem.sfs_theory
get_sfs_terms	lunadem.sfs_theory
get_test_scene_manifest	lunadem.datasets.kaguya
ground_coverage_meters	lunadem.geometry.conversions
km_to_m	lunadem.geometry.conversions
latlon_to_cartesian_moon	lunadem.geometry.conversions
line_time_seconds	lunadem.geometry.conversions
list_available_rovers	lunadem.landing.rovers
list_kaguya_scenes	lunadem.datasets.kaguya
list_model_names	lunadem.ml.inference
load_camera_model	lunadem.datasets.kaguya
load_image	lunadem.io.image
load_kaguya_scene	lunadem.datasets.kaguya
load_pds_label	lunadem.io.pds
load_stac_item	lunadem.datasets.kaguya
m_to_km	lunadem.geometry.conversions
m_to_mm	lunadem.geometry.conversions
mean_vector	lunadem.geometry.conversions
mm_to_m	lunadem.geometry.conversions
normalize_to_unit_range	lunadem.utils.arrays
path_length	lunadem.geometry.conversions
pixel_scale_meters	lunadem.geometry.scaling
pixel_to_projected	lunadem.geometry.conversions
plot_3d_surface	lunadem.visualization.plots
plot_3d_surface_interactive	lunadem.visualization.interactive
plot_depth_map	lunadem.visualization.plots
plot_landing_site_2d	lunadem.visualization
plot_landing_site_3d	lunadem.visualization
plot_moon_surface_3d	lunadem.visualization
plot_scene_geometry_3d	lunadem.visualization
predict_illumination	lunadem.ml.inference
predict_scene_location	lunadem.ml.inference
predict_scene_metadata	lunadem.ml.inference
predict_view_geometry	lunadem.ml.inference
projected_to_pixel	lunadem.geometry.conversions
rad_to_deg	lunadem.geometry.conversions
save_dem_as_geotiff	lunadem.io.raster
save_dem_as_obj	lunadem.io.mesh
save_dem_as_ply	lunadem.io.mesh
save_manifest	lunadem.io.manifest

Function / Class	Source Module
scale_dem_to_meters	lunadem.geometry.scaling
scene_geometry_to_dict	lunadem.datasets.kaguya
summarize_scene_metadata	lunadem.datasets.kaguya

lunadem v0.3.0

Official Python Library Documentation

Author & Developer: **Kartavya Mahesh Suryawanshi**

GitHub: github.com/Kartavya728

Documentation Author: Kartavya Mahesh Suryawanshi

2026-04-13